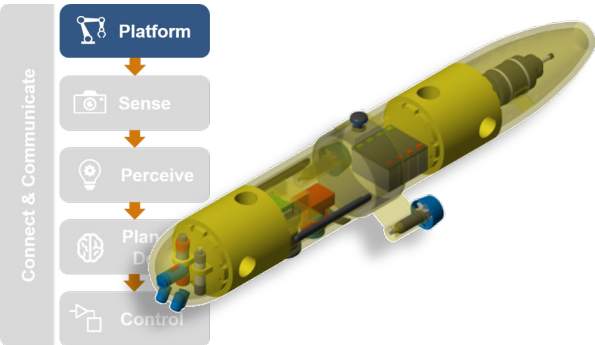


Modeling the Underwater Vehicle Platform

Last updated: April 14th, 2021

A reasonably accurate dynamic model of the vehicle platform is an essential tool for the development and evaluation of any complex guidance and navigation control algorithms for an autonomous system. In addition, a good model will give us the ability to test and verify system-level and component-level requirements using desktop simulation, and allow us to find and fix errors early in the design process, which will result in significantly reduced risks and fewer issues when moving either to prototypes or production hardware.



This section presents an overview of some of the basic capabilities in **SimScope** for modeling and simulation of multi-domain physical systems. In particular, the outlined workflow focuses on **SimScope Multibody** for modeling the mechanical dynamics, and its integration with **SimScope** and **SimScope Electrical** for modeling the propulsion actuators as well as the electric power distribution system of the autonomous underwater vehicle platform.

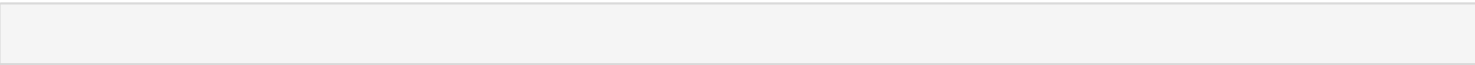


Table of Contents

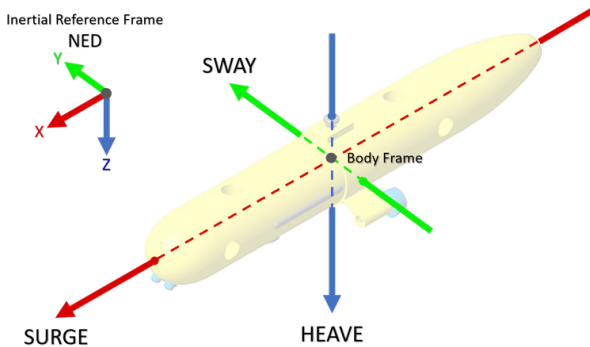
Underwater vehicle dynamics.....	2
Fundamental mechanical modeling techniques.....	2
Create mechanical models using rigid body primitives.....	3
Explore parameterized designs.....	3
Model joints, connections and external forces and torques.....	4
Create mechanical models using part files from CAD.....	4
Add tracking cameras in the mechanics explorer.....	5
Import complete mechanical assemblies from CAD.....	5
Compute inertial properties and hydrostatic forces.....	6
Apply hydrostatic forces - gravity and buoyancy effects.....	6
Apply hydrodynamic forces - drag, lift, and added mass effects.....	7
Compare and contrast mechanical modeling implementations.....	8
Underwater vehicle electrical distribution and propulsion.....	9
Electrical circuit modeling basics.....	9
Model generic electrical actuators.....	10

Model simplified electric motor drives.....	10
create custom components.....	11
Model propellers and thrusters.....	11
Interface with the 3D mechanical model.....	12
estimate battery model parameters using test data.....	13

Underwater Vehicle Dynamics

AN appropriate dynamic model for a fully autonomous underwater vehicle platform needs to include the motion of the vehicle in six degrees of freedom, gravitational and buoyancy effects, hydrodynamic drag and lift forces generated by the vehicle body moving in water, added mass effects induced by the accelerations of the water displaced by the vehicle, disturbance forces from underwater currents, and the propulsive forces from the main propellers and tunnel thrusters.

the model will conform to a North-East-Down (NED) inertial frame of reference convention, and align a non-inertial body frame, attached to the center of mass of the vehicle, with the surge (longitudinal), sway (lateral) and heave (vertical) principal axes of the vehicle body.

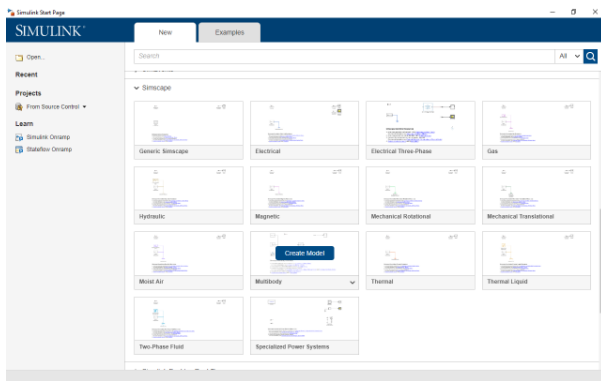


We will develop the mathematical model of the vehicle in Simulink using Simscape Multibody, which provides a specialized environment for modeling and simulating complex 3D mechanical systems.

At its core, Simscape Multibody treats any mechanism as a combination of interconnected bodies (inertias), joints and constraints (relative DOFs), and force elements, and uses that approach to formulate and solve the dynamics for the complete system without the user having the need to explicitly derive any equations of motion.

Fundamental mechanical modeling techniques

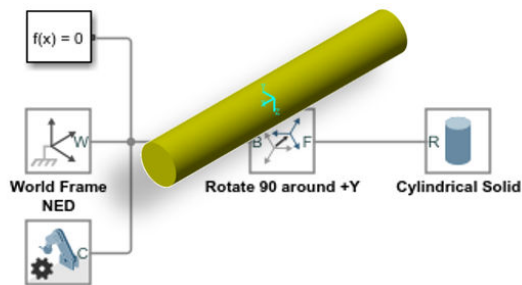
To get started with a new mechanism, open the pre-made template for a new Simscape Multibody model. This pre-made template is located under the Simscape section on the [Simulink Start Page](#).



```
smnew;
```

Create mechanical models using rigid body primitives

the **Body Elements** block library provides a variety of preset shapes like bricks, cylinders, extrusions, etc. custom shapes can also be imported from external files in STEP or STL format. Body frames can be rigidly connected using **Rigid Transform** blocks to form compound rigid bodies.

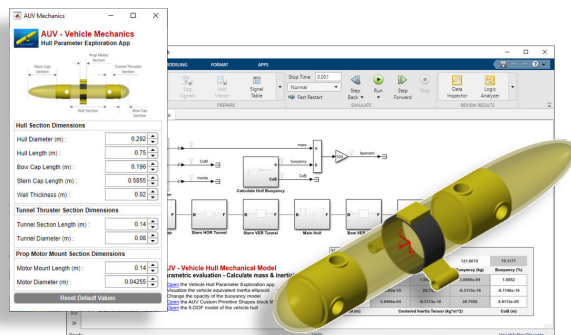


Rigid Transform blocks can also be used to create arbitrary frame rotations and translations to align bodies with the world frame or relative to each other, and to line up frames with the appropriate degrees of freedom.

```
open_system('AUV_MechanicsStartNew');
```

Explore parameterized designs

AN IMPORTANT ADVANTAGE OF BEING ABLE TO USE RIGID BODY PRIMITIVES, ESPECIALLY IN THE INITIAL EXPLORATORY STAGES OF THE DESIGN - WHEN CAD DRAWINGS OF THE MECHANICAL PARTS MIGHT NOT YET BE AVAILABLE - IS THAT THESE PARTS CAN BE FULLY PARAMETERIZED USING MATLAB VARIABLES AND EXPRESSIONS.

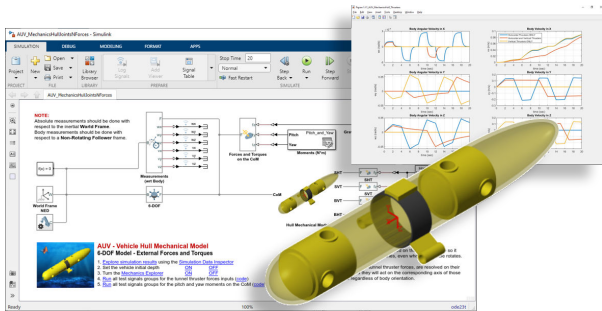


this means you can use simple MATLAB scripts or create customized graphical user interfaces with the MATLAB App Designer to modify any parameters in the mechanical model and quickly evaluate the effect of those parameter changes in the preliminary design.

```
open_system('AUV_MechanicsHullParametric');
```

Model joint connections and external forces and torques

JOINT blocks connect the frames of two rigid bodies and impose the primary kinematic constraints that determine how these bodies can move relative to each other. JOINT blocks are parameterized in terms of the degrees of freedom they provide. these DOFs range from zero in the **Weld JOINT** block to six - three translational and three rotational - in the **6-DOF JOINT** and **Bushing JOINT** blocks. All JOINTS provide the means for actuating (with either forces and/or torques or motion profiles) and sensing their corresponding DOFs.



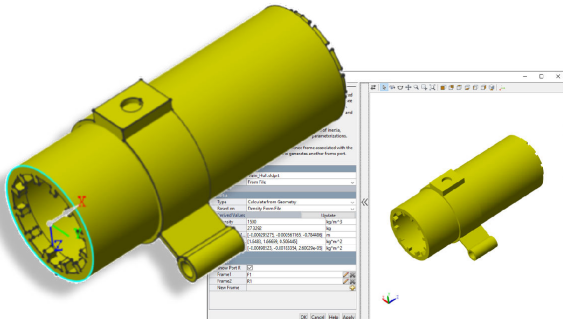
the **EXTERNAL FORCE AND TORQUE** block can be used to apply forces and/or torques to a connected rigid body frame. the block provides options to select the resolution frame - world or attached frame - for the exertion of each external force and torque.

the **TRANSFORM SENSOR** block provides measurements of the spatial relationship between any two frames in the Multibody Model. these measurements are resolved with respect to a chosen measurement frame - measurement resolution options include world as well as rotating and non-rotating base and follower frames.

```
open_system('AUV_MechanicsHullJointsNForces');
```

Create mechanical models using part files from CAD

the **FILE SOLID** block associates individual part files from most standard CAD formats to body blocks in the Simscape Multibody Model. Geometry and material properties are read directly from the part file or can be customized on the block mask.

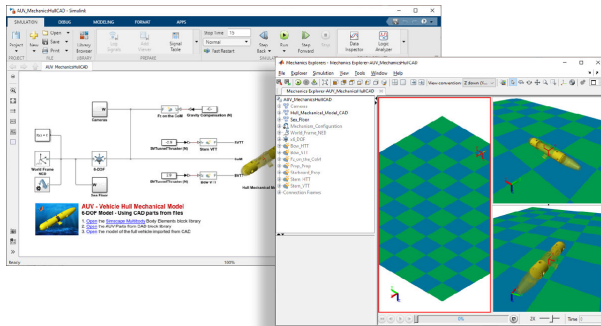


new body frames can be defined on the 3D interface and added to the part. the new frames can be positioned and oriented relative to existing reference frames or geometric features of the solid.

```
open_system('AUV_MechanicsStartNewCAD');
```

Add tracking cameras in the Mechanics Explorer

cameras define the model viewpoints used during animation playback. the **Mechanics Explorer** supports two camera types - global and dynamic. the global camera provides a static viewpoint that you can manipulate interactively during animation and playback. dynamic cameras can be attached to any frame in the model, and provide moving viewpoints that can be predefined using the **camera manager** and enable the tracking of any moving part in the system.

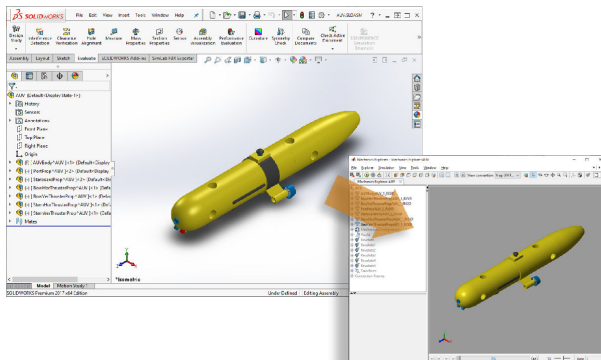


```
open_system('AUV_MechanicsHullCAD.slx');
```

Import complete mechanical assemblies from CAD

full mechanical assemblies, including all masses, inertias, maps, and 3D geometry can be translated directly from multiple CAD packages (including SolidWorks®, PTC® Creo and Autodesk Inventor®) and automatically imported into a model using the **Simscape Multibody Link** CAD plug-in. this plug-in can be downloaded for free from the **Simscape Multibody Link download page**.

open the autonomous underwater vehicle SolidWorks® assembly file, and use the **Simscape Multibody Link** export function under the tools menu. if SolidWorks® is not available, the translation process can be reviewed in this **step by step video**.

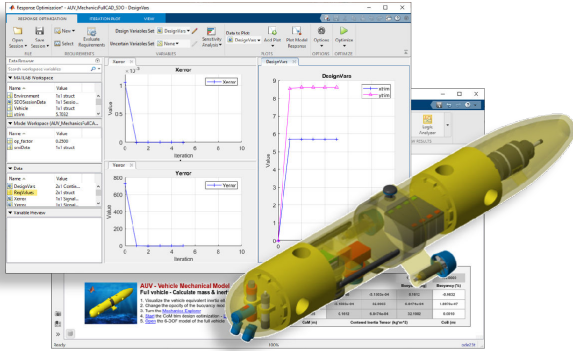


AFTER THE CAD TRANSLATION IS FINISHED AND THE CORRESPONDING XML FILE IS GENERATED, THE ASSOCIATED SIMSCAPE MULTIBODY MODEL CAN BE AUTOMATICALLY CREATED USING THE `smimport()` FUNCTION.

```
smimport('AUV.xml')
```

Compute inertial properties and hydrostatic forces

THE **INERTIA SENSOR** BLOCK PROVIDES THE MEANS TO MEASURE THE INERTIAL PROPERTIES - MASS, CENTER OF MASS, INERTIA TENSORS - FOR INDIVIDUAL OR COLLECTIONS OF BODY ELEMENTS IN A SIMSCAPE MULTIBODY MODEL.

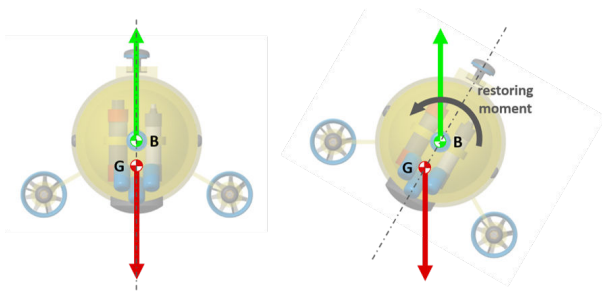


ONCE THE COMPLETE MECHANICAL MODEL OF THE VEHICLE - INCLUDING ALL MASS AND INERTIAL CONTRIBUTIONS FROM SENSORS AND ELECTRONIC COMPONENTS - HAS BEEN IMPLEMENTED IN SIMSCAPE MULTIBODY, WE CAN FINE TUNE THE LOCATION OF THE RESULTING CENTER OF MASS BY PERFORMING A DESIGN OPTIMIZATION DIRECTLY ON THE SIMULINK MODEL USING THE **RESPONSE OPTIMIZATION** APP.

```
open_system('AUV_MechanicsFullCAD_SDO');
```

Apply hydrostatic forces - gravity and buoyancy effects

IN ORDER TO MAINTAIN STABLE EQUILIBRIUM, A FULLY SUBMERGED UNDERWATER VEHICLE MUST HAVE THE CENTER OF GRAVITY SITUATED DIRECTLY BELOW THE CENTER OF BUOYANCY. THIS WILL GENERATE A RESTORING MOMENT THAT COUNTERS ANY EXTERNAL FORCES OR DISTURBANCES THAT PRODUCE ROTATIONS AROUND THE BODY ROLL AND/OR PITCH AXES AND WILL HELP BRING THE VEHICLE BACK TO A LEVELLED ORIENTATION.



NOTE THAT BOTH THE GRAVITATIONAL FORCE AND THE BUOYANCY FORCE ARE ALWAYS ALIGNED WITH THE VERTICAL AXIS OF THE WORLD REFERENCE FRAME, AND THE RESTORING MOMENT WILL ALWAYS ACT TO BRING BACK THE CENTER OF GRAVITY TO ITS ORIGINAL POSITION BELOW THE CENTER OF BUOYANCY.

```
open_system('AUV_MechanicsFullHydrostatics');
```

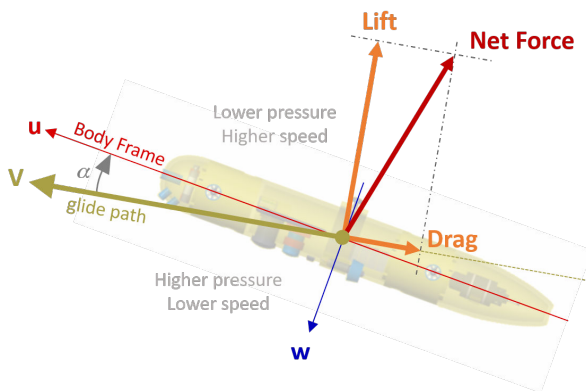
Apply hydrodynamic forces - drag, lift, and added mass effects

the fundamental governing equation for the dynamics of the underwater vehicle is of the form:

$$(M_{RB} + M_A)\dot{\nu} + (C_{RB}(\nu) + C_A(\nu))\nu + D(\nu)\nu + g(\eta) = F_{propulsion}$$

where $\nu = [V_b \ \omega_b]^T = [u \ v \ w \ p \ q \ r]^T$ is the body-fixed velocity vector, $\eta = [x \ y \ z \ \phi \ \theta \ \psi]^T$ is the earth-fixed position and orientation vector, M_{RB} and $C_{RB}(\nu)$ are the mass matrix and control effects associated with the rigid body, M_A is the added mass matrix, $C_A(\nu)$ is the control effect induced by the added mass, $D(\nu)$ is the damping matrix that includes all the drag and lift forces, and $g(\eta)$ represents the combined gravitational and buoyancy effects. Note that Simscape Multibody will take care of implementing the rigid body dynamics as well as account for the gravitational forces. All other effects will need to be applied as external forces and moments.

Drag and lift forces are the result of pressure and friction due to fluid flowing around the surface of the vehicle. Lift is perpendicular to the flow direction and drag is parallel and opposite the flow direction. Both of these forces are a function of the square of the velocity, and are highly dependent on the shape of the vehicle, the transitions between laminar and turbulent regimes in the flow defined by the Reynolds Number, the skin friction, the distance to the ocean bottom, and specifically the incidence angle α (the inclination of the body with respect to direction of the flow) and the side slip angle β (the lateral deviation from the glide path) among many other complex factors.

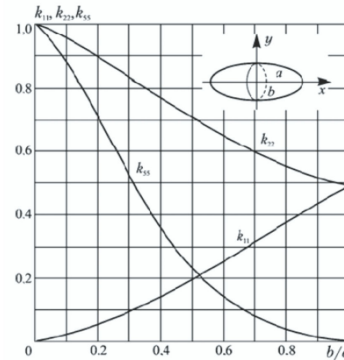
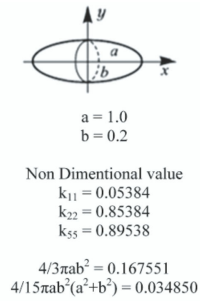
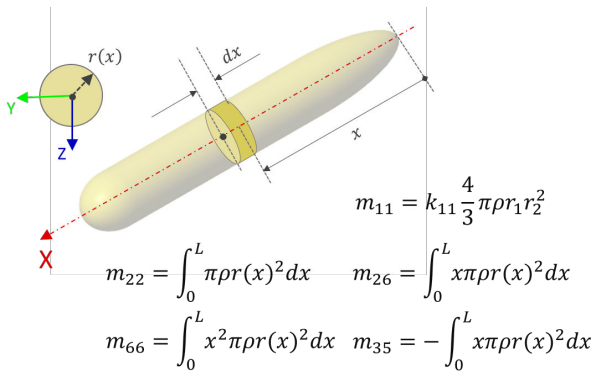


$$F_{drag} = \frac{1}{2} \rho V^2 C_d A \quad F_{lift} = \frac{1}{2} \rho V^2 C_l A$$

where ρ is the density of the fluid, V is the vehicle velocity and A is a cross section reference area. Drag and lift coefficients C_d and C_l are normally obtained from laboratory experiments and used in practice to capture and characterize the many additional complex dependencies between the fluid and the surface of the vehicle.

Added mass is a pressure induced force and moment on the body resulting from the inertial characteristics of the volume of surrounding fluid that must be moved as the vehicle accelerates or decelerates. This effect is significant for bodies moving in water, and its contribution can be modeled as an apparent increase in the actual mass of the underwater vehicle.

IN GENERAL, COMPUTING THE ADDED MASS MATRIX COEFFICIENTS FOR A GENERIC SHAPE, IS A COMPLEX PROBLEM THAT NEEDS TO BE SOLVED USING CFD METHODS AND VALIDATED EXPERIMENTALLY. HOWEVER, THERE EXIST SEVERAL SIMPLIFIED METHODS THAT PROVIDE APPROPRIATE FIRST ORDER APPROXIMATIONS FOR COMMONLY USED SHAPES (I.E. SPHERES, CYLINDERS, PLATES, ETC.)



FOR OUR EXAMPLE, THE HULL OF THE VEHICLE IS A TYPICAL ELONGATED MYRING SHAPE, SO WE WILL ASSUME ROTATIONAL SYMMETRY AROUND THE X-AXIS AND APPLY A COMBINATION OF SLENDER BODY THEORY AND LAMB'S K-FACTORS FOR A PROLATE SPHEROID TO ESTIMATE THE COEFFICIENTS OF THE ADDED MASS MATRIX, WHICH WILL THEN BE REDUCED TO THE FORM:

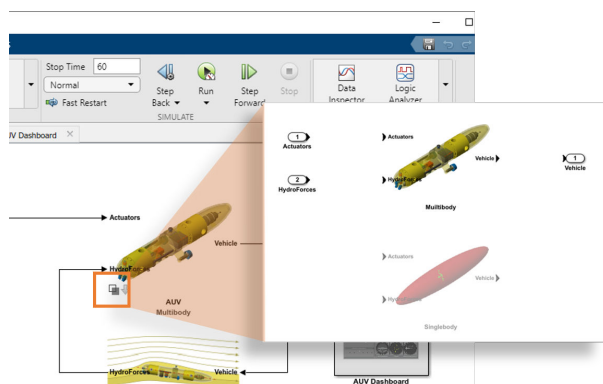
$$M_A = \begin{bmatrix} m_{11} & 0 & 0 & 0 & 0 & 0 \\ 0 & m_{22} & 0 & 0 & 0 & m_{26} \\ 0 & 0 & m_{33} & 0 & m_{35} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & m_{53} & 0 & m_{55} & 0 \\ 0 & m_{62} & 0 & 0 & 0 & m_{66} \end{bmatrix}$$

FOR UNDERWATER VEHICLES THESE COEFFICIENTS ARE USUALLY TAKEN AS CONSTANTS AND GIVEN THE ASSUMPTION OF TOP-DOWN AND PORT-STARBOARD SYMMETRY WE CAN ALSO MAKE: $m_{22} = m_{33}$ AND $m_{55} = m_{66}$.

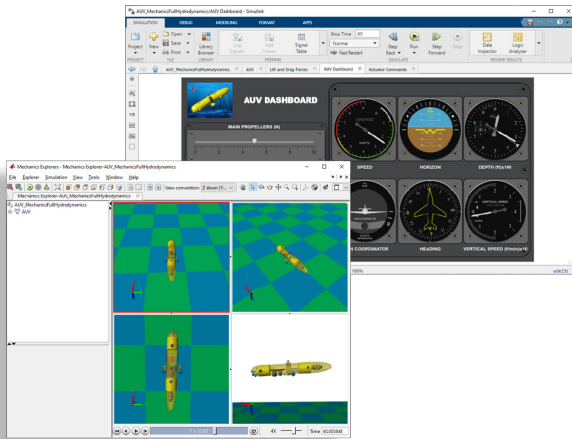
```
open_system('AUV_MechanicsFullHydrodynamics');
```

Compare and contrast mechanical modeling implementations

SUBSYSTEM VARIANTS IN SIMULINK PROVIDE AN EFFICIENT WAY TO INCLUDE DIFFERENT IMPLEMENTATIONS FOR INDIVIDUAL PARTS OF A SINGLE MODEL. ONLY ONE OF THESE IMPLEMENTATIONS CAN BE ACTIVE AT A TIME, AND THE CHOICE OF THE ACTIVE VARIANT, OR THE ACTIVE COMBINATION OF VARIANTS, CAN BE EASILY CONTROLLED WITH A MATLAB PARAMETER OR EXPRESSION.



we will leverage the single body 6DOF (Zuler Angles) block from the equations of motion library in the Aerospace Blockset to provide an alternative to our multibody representation of the underwater vehicle rigid body dynamics. this will allow us to compare and contrast both implementations against each other and give us the flexibility to choose the more appropriate variant to use for a given simulation task.

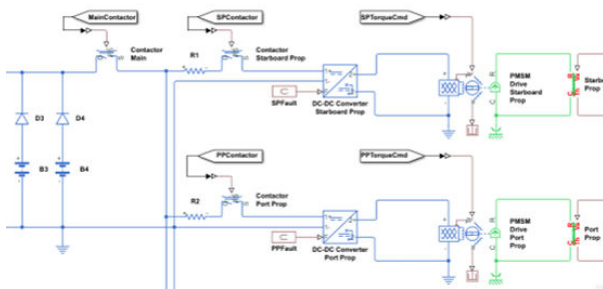


in addition, we can take advantage of the collection of standard cockpit dashboard displays included in the flight instruments library of the Aerospace Blockset. even though the use of these displays is aimed at more traditional aircraft applications, they can be adapted to monitor the status of our underwater vehicle during simulation.

```
open_system('AUV_MechanicsFullHydrodynamicsVAR');
```

Underwater Vehicle Electrical Distribution and Propulsion

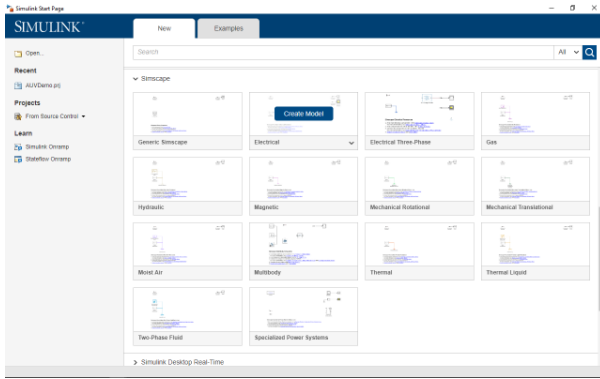
Battery life and total power consumption are crucial factors on the endurance and mission capabilities for an autonomous underwater vehicle. incorporating the dynamics of the battery, the electric motors, and the distribution of power to the on-board sensors, electronics, and communication devices into our system-level model will enable us to simulate the complete vehicle platform, and help us avoid possible integration issues later in the design process.



for this task, we will use Simscape and Simscape Electrical. these libraries provide components that can be assembled into schematic diagrams and allow rapid model creation of electro-mechanical systems that directly integrate with block diagrams and other modeling constructs in the Simulink environment.

Electrical circuit modeling basics

to get started with a new electrical circuit, open the pre-made template for a new Simscape Electrical Model. this pre-made template is located under the Simscape section on the [Simulink Start Page](#).

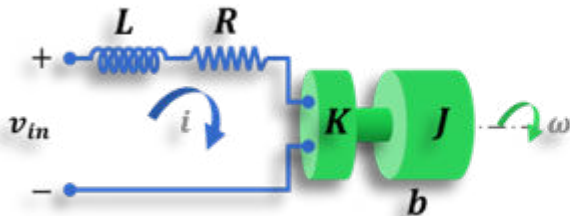


```
ssc_new('','electrical');
```

Model generic electrical actuators

A motor is a general purpose electro-mechanical actuation device that converts electrical energy into rotational mechanical energy - producing a torque that is proportional to its armature current and a corresponding voltage drop proportional to its angular velocity.

the equivalent electrical circuit for a simple DC motor is represented by an armature resistor and an armature inductor connected in series. the equivalent mechanical circuit is represented by a rotor inertia and usually some internal rotor damping. these dynamics can be represented by two first order differential equations:



Loop equation (voltage balance):

$$0 = v_{in} - L \frac{di}{dt} - Ri - K_{emf} \omega$$

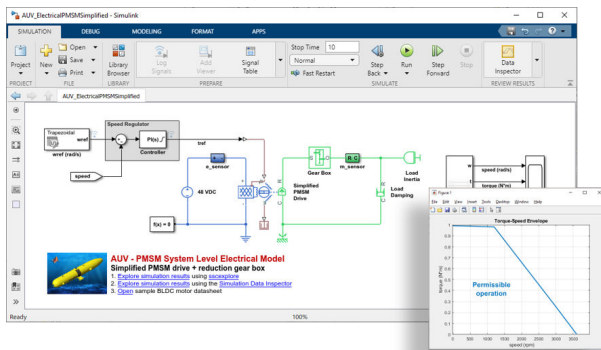
Node equation (torque balance):

$$0 = K_t i - J \frac{d\omega}{dt} - b\omega$$

```
open_system('AUV_ElectricalDCmotorFoundation');
```

Model simplified electric motor drives

the [Simplified FMSM Drive](#) block represents a brushless dc motor model with closed-loop torque control. this block abstracts the torque-speed behavior of the combined motor and motor driver in order to support system-level simulation where simulation speed is important.



Note that the **Electromechanical** libraries in Simscape Electrical contain a number of pre-made component models for a variety of motor types: from generic linear and rotary actuators, to induction machines, to stepper motors, to brushless dc and permanent magnet synchronous motors.

A variety of application examples for all these different motor types can be found in the documentation. For a more detailed example model of a speed controller for a BLDC motor drive, including the commutation logic and the transistor gate controls for a three-phase IGBT inverter, refer to [this](#) documentation page.

```
open_system('AUV_ElectricalPMSMSimplified');
```

Create custom components

With the MATLAB based **Simulink Language** you can create custom physical components, domains and libraries. Custom Simulink files (.ssc) can be included in a model diagram by using the **Simulink Component** block, or alternatively the command `ssc_build` automatically creates a Simulink library file containing the sublibraries and blocks generated from all the Simulink files in a specified package directory.

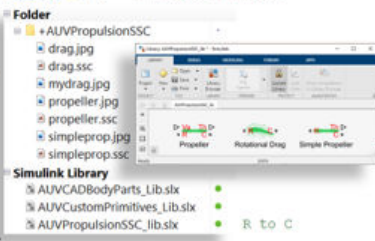
The Simulink Language supports a variety of constructs and a large number of functions and allows the user to define the desired mathematical relationships between the component variables.

```
parameters
    Cd = { 1e-6, '(N*m)*(s/rad)^2' }; % Drag coefficient
end

variables
    t = { 0, 'N*m' };
    w = { 0, 'rad/s' };
end

branches
    t: R.t -> C.t;
end

equations
    assert(Cd>=0)
    w = R.w - C.w;
    t = sign(w)*Cd*w^2;
end
```



As an example, we slightly modified the code of the original Foundation Library **Rotational Damper** to implement a custom **Rotational Drag** component.

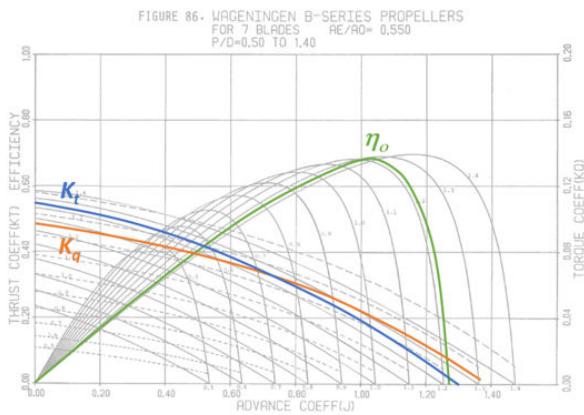
```
open_system('AUV_ElectricalCustomDrag');
```

Model propellers and thrusters

the thrust and torque characteristics of the propellers and thrusters will have significant influence in the performance of the underwater vehicle. It is critical that these characteristics be properly represented in any dynamic simulation model of the platform.

thrust and torque have complex nonlinear relationships that are highly dependent on the propeller geometry - diameter, pitch or advance ratio, blade area, blade thickness, etc. - Number of blades, rotational speed, inflow velocity of the fluid and Reynolds Number among many other factors. To avoid this complexity we will take advantage of commonly used propeller efficiency curves (Wageningen B-series Propellers) to extract the values of the thrust and torque coefficients K_t and K_q , as well as the open water efficiency η_o for our calculations.

For a given number of blades and a given expanded area ratio - i.e. the ratio of the blade area outside of the hub with respect to the overall propeller disk area - the charts provide the values for these coefficients as a function of the pitch to diameter ratio and the advance velocity coefficient $J = V_a/ND$.



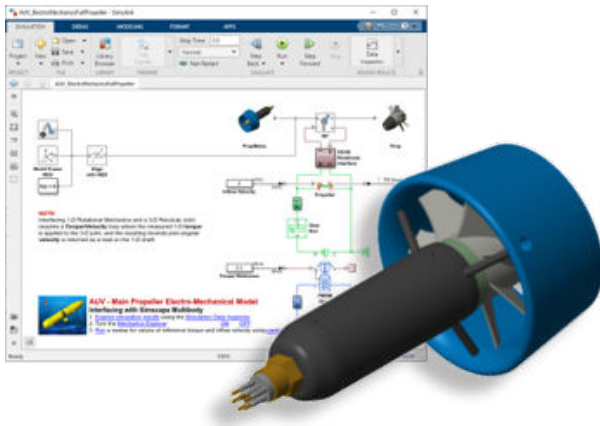
$$T = K_t \rho N^2 D^4 \quad Q = K_q \rho N^2 D^5$$

where T is the propeller axial thrust force and Q is the propeller shaft torque, ρ is the density of the fluid, N is the propeller rotational speed in rev/sec and D is the propeller diameter.

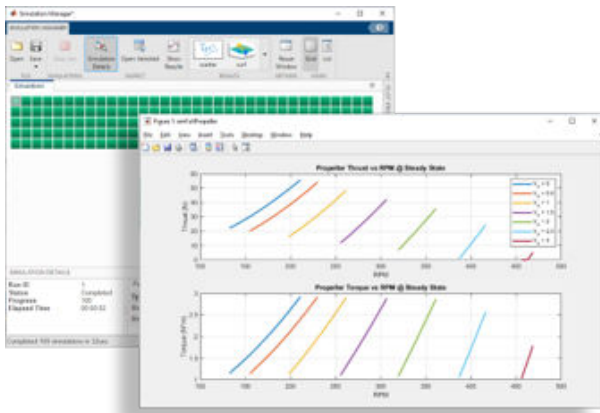
```
open_system('AUV_ElectroMechanicsSSCPropeller');
```

Interface with the 3D mechanical model

In order to directly connect the electric motor and the propeller dynamics into the 6-DOF vehicle model, it is necessary to create an interface between the Simscape 1-D Rotational Mechanics domain and the 3-D Revolute Joint in Simscape Multibody using physical signals.



this rotational interface requires a torque/velocity loop where the measured 1-D torque is applied to the 3-D joint, and the resulting revolute joint angular velocity is returned as a load on the 1-D shaft.



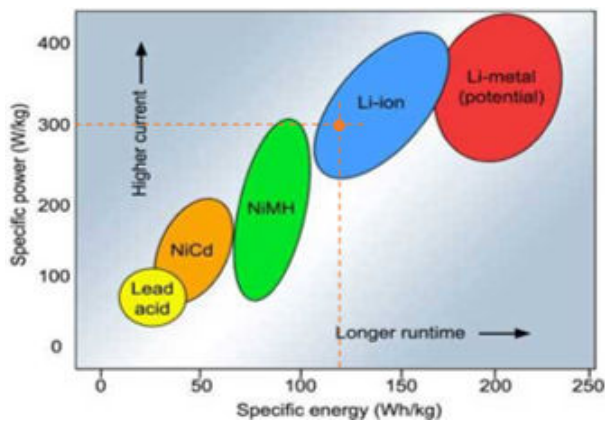
we can use this example model to evaluate the propeller response to a range of reference torque and inflow velocity values at steady state. Note that when running a large number of tests or multiple parameter sweeps, you can leverage the [Parallel Computing Toolbox](#) to automatically distribute the individual simulations through a parallel pool of workers on your local machine using the functionality in [parsim\(\)](#).

```
open_system('AUV_ElectroMechanicsFullPropeller');
```

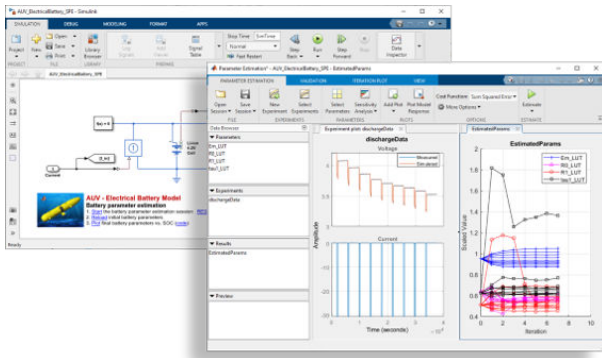
Estimate battery model parameters using test data

Given that the battery is the only source of power available for the autonomous operation of the vehicle, we need to ensure that the model of the component accurately captures its power delivery capability and its voltage discharge characteristics.

Li-ion batteries are presently the preferred chemistry choice for underwater vehicle applications because of their high energy and power density, good performance at low temperatures and comparatively small form factors.



Simulink Electrical provides two pre-configured options for modeling batteries. A first principles based [Battery](#) block that uses a behavioral approach and a [table-based Battery](#) block suitable for matching experimental data. Both of these blocks include the ability to add thermal characteristics and other effects to the model if needed.



The [Parameter Estimation](#) App allows us to set up and run a multi-variable optimization to estimate the parameters in a Simulink model using measured data. In our example, we will use this tool to adjust the battery parameters such that the component model matches the discharge curve obtained from an experiment. For a more detailed example of extracting battery parameters from data, including tests for multiple operating temperatures, refer to [this](#) documentation page.

```
open_system('AUV_ElectricalBattery_SPE');
```

Copyright 2020 The MathWorks, Inc.

[Prev: Main Menu](#)